Building a Data Diode: Quick Start

A one-way syslog link in an afternoon

Alin-Adrian Anton
alin.anton@upt.ro
Politehnica University of Timişoara

Companion to *The Data Diode Handbook*
Anton, Csereoka, Capotă & Cioargă, *Sensors* **24**(20), 6537 (2024)

Abstract

This is the short road. It gets you from an empty bench to syslog messages crossing a genuinely unidirectional link, running the formally verified ld_amp and ld_deamp daemons at each end. Two routes are given: a **software diode** for the bench — fast, free, and *not for production* — and a **hardware diode** built from three off-the-shelf media converters, which is the real thing. Every claim about one-wayness is paired with a test that proves it. For the reasoning behind the choices, the optical power budgets, the two-converter/splitter variant and the threat model, see the companion *Data Diode Handbook*.

Contents

1 What you are building

A data diode carries information from a lower-security network to a higher-security one and makes the reverse physically impossible. Logs are the classic application: the Security Operations Centre must see the plant's logs, but nothing in the corporate world should be able to reach —

or write to — the plant. That is a Bell–LaPadula arrangement, and a diode enforces it with optics rather than with policy.

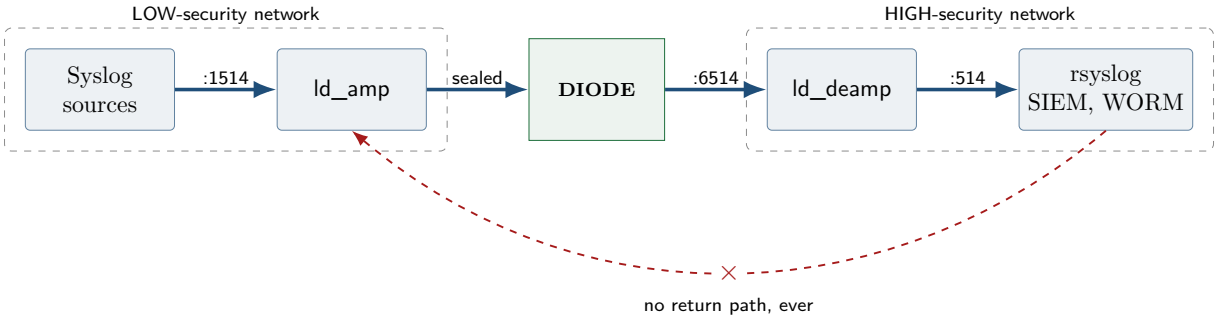

Figure 1: The shape of the system. Everything left of the diode is untrusted; everything right of it can never be reached from the left.

Because there is no return path there can be no acknowledgement, so there is no TCP, no ARP resolution, no DNS and no retransmission. The transport must be UDP, and reliability has to come from sending redundancy *ahead of time* rather than asking for a repeat. That is exactly what ld_amp and ld_deamp do: forward error correction plus authenticated encryption, with the parsing and reconstruction path machine-checked free of run-time errors.

Which route should I take?

Take the **software route** (§3) if you want to exercise ld_amp/ld_deamp today, write tests, or demonstrate the pipeline. Take the **hardware route** (§4) for anything that protects a real network. The software route is a bench instrument, not a security control — see the warning at the head of §3.

2 Prerequisites

- Two GNU/Linux hosts (or two VMs, or two network namespaces), each with a spare Ethernet interface for the diode link. In this guide the LOW host is `192.168.10.1` and the HIGH host is `192.168.10.2`, both on `enp1s0`.
- log-diode built: `./tools/build.sh` produces `bin/ld_amp` and `bin/ld_deamp`. This needs GNAT 14.2.0 and gprbuild; see `tools/env.sh`.
- Root on both hosts.
- For the hardware route: three media converters, three simplex fiber strands, two Ethernet cables, and three power supplies. Bill of materials in §4.2.

Generate the shared key now — you will need the same 64 hex characters on both sides:

```
openssl rand -hex 32
# e.g. 9f2c...4ab1   (32 bytes = 64 hex chars, ChaCha20-Poly1305)
```

The key cannot be exchanged over the diode

There is no back channel, so there is no key agreement protocol available. The key must be carried across by hand — USB token, printed sheet, an existing out-of-band management channel. Plan this before you rack anything.

3 Route A — the software diode (bench only)

Read this before using anything in this section

A firewall-based diode enforces one-wayness with a *mutable ruleset inside a kernel that the attacker may be able to reach.* A root compromise, a `nft flush ruleset`, a misapplied configuration-management run, or a kernel bug restores the return path instantly and silently. It is a fine instrument for developing against ld_amp/ld_deamp, for continuous integration, and for teaching. **Do not deploy it as the boundary between two security domains.** Use §4.

3.1 Link-layer setup (both hosts)

With no return path there is no ARP resolution, so the peer's MAC address has to be configured by hand. This is the arrangement described in the paper (§2.2) and it is needed for the hardware diode too.

On the LOW host, record the HIGH host's MAC:

```
bb:bb:bb:bb:bb:bb 192.168.10.2
```

Listing 1: `/etc/ethers` on the LOW host

and on the HIGH host, record the LOW host's MAC:

```
aa:aa:aa:aa:aa:aa 192.168.10.1
```

Listing 2: `/etc/ethers` on the HIGH host

Then, on each host respectively:

```
# LOW host
ip addr add 192.168.10.1/24 dev enp1s0
# HIGH host
ip addr add 192.168.10.2/24 dev enp1s0

# both hosts: load the static entries and make them permanent
arp -f /etc/ethers
ip neigh show dev enp1s0        # entries must read PERMANENT
```

Make it survive a reboot with a unit file (both hosts):

```
[Unit]
Description=Static ARP and addressing for the diode link
After=network-pre.target
Wants=network-pre.target

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/arp -f /etc/ethers

[Install]
WantedBy=multi-user.target
```

Listing 3: `/etc/systemd/system/diode-link.service`

```
systemctl enable --now diode-link.service
```

Silence the interface so it cannot answer anything (both hosts):

```
net.ipv6.conf.enp1s0.disable_ipv6 = 1
net.ipv4.conf.enp1s0.arp_ignore   = 8
net.ipv4.conf.enp1s0.arp_announce = 2
net.ipv4.conf.enp1s0.accept_redirects = 0
net.ipv4.conf.enp1s0.send_redirects   = 0
net.ipv4.conf.enp1s0.accept_source_route = 0
```

Listing 4: `/etc/sysctl.d/90-diode.conf`

```
sysctl --system
```

3.2 The ruleset

The single most important rule is negative: **no connection tracking on the diode interface**. A conntrack state entry, an `ESTABLISHED,RELATED` accept, or a pf `keep state` *is* a return path. So is an ICMP port-unreachable and a TCP RST.

nftables (the modern default). On the LOW host, egress only:

```
#!/usr/sbin/nft -f
flush ruleset

define DIODE_IF = "enp1s0"
define PEER     = 192.168.10.2
define PORT     = 6514

# Drop every frame arriving on the diode NIC, at the earliest possible
# point -- this catches ARP, LLDP and IPv6 too, which the inet family
# input hook would not see.
table netdev diodeguard {
    chain ingress {
        type filter hook ingress device enp1s0 priority -500; policy drop;
    }
}

table inet diode {
    chain input {
        type filter hook input priority filter; policy drop;
        iif lo accept
        iifname $DIODE_IF drop
    }
    chain output {
        type filter hook output priority filter; policy drop;
        oif lo accept
        oifname $DIODE_IF ip daddr $PEER udp dport $PORT accept
        oifname $DIODE_IF drop
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
    }
}
```

Listing 5: `/etc/nftables.conf` — LOW host

On the HIGH host, ingress only:

```
#!/usr/sbin/nft -f
flush ruleset

define DIODE_IF = "enp1s0"
```

```
define PEER     = 192.168.10.1
define PORT     = 6514

table inet diode {
    chain input {
        type filter hook input priority filter; policy drop;
        iif lo accept
        iifname $DIODE_IF ip saddr $PEER udp dport $PORT accept
        iifname $DIODE_IF drop
    }
    chain output {
        type filter hook output priority filter; policy drop;
        oif lo accept
        # Nothing may ever leave by the diode NIC.
        oifname $DIODE_IF drop
    }
    chain forward {
        type filter hook forward priority filter; policy drop;
    }
}
```

Listing 6: `/etc/nftables.conf` — HIGH host

```
systemctl enable --now nftables
nft list ruleset          # read it back and check it is what you wrote
```

iptables and pf. Equivalent rulesets for legacy iptables, OpenBSD pf and FreeBSD pf — including the `set block-policy drop` subtlety that stops pf from answering with RSTs — are in the Handbook, §4.

3.3 Route A′ — a unidirectional layer-2 tunnel

There is a better software option than filtering at layer 3. Bridge a TAP interface to each of two *real* Ethernet segments and forward frames between the taps with a program that has no reverse code path. You get the same Ethernet semantics as the optical diode of §4 — a one-way layer-2 tunnel — with software in place of the missing transmitter.

```
ip tuntap add dev tapdlow  mode tap
ip tuntap add dev tapdhigh mode tap

# One bridge per side.  STP off -- BPDUs are bidirectional signalling.
ip link add br-low  type bridge stp_state 0
ip link add br-high type bridge stp_state 0
ip link set eth0     master br-low     # NIC facing the LOW segment
ip link set tapdlow  master br-low
ip link set eth1     master br-high    # NIC facing the HIGH segment
ip link set tapdhigh master br-high
for i in eth0 eth1 tapdlow tapdhigh br-low br-high; do ip link set "$i" up; done

./tapdiode tapdlow tapdhigh            # forwards low -> high, and only that
```

Listing 7: On the forwarding host

The forwarder is about a hundred lines of C; the whole program, with notes on its design, is in the Handbook (§4.7). Its core is the part worth seeing here: only the input descriptor is ever polled, and `read()` is never called on the output descriptor, so the reverse direction is absent rather than forbidden.

```
1 pfd.fd = in_fd;          /* the ONLY descriptor ever polled */
2 pfd.events = POLLIN;
```

```c

while (!stop_requested) {
    if (poll(&pfd, 1, -1) < 0) { if (errno == EINTR) continue; break; }

    n = read(in_fd, frame, sizeof frame);        /* only ever in_fd  */
    if (n <= 0) { if (errno == EINTR) continue; break; }

    if (write(out_fd, frame, (size_t) n) != n)   /* only ever out_fd */
        lost++;                                  /* loss, never feedback */
}
```

Listing 8: The loop, in essence

Two bridges, never one

Everything rests on `br-low` and `br-high` being separate. Putting `eth0` and `eth1` in the *same* bridge gives an ordinary bidirectional bridge and destroys the property — and it is one command. Assert it afterwards: `bridge link show` must never list both NICs under one master. This is still software, so §4 remains the answer for a real boundary.

Verify at layer 2 — `ping` proves nothing about frames the IP stack never generates. Sniff `'ether proto 0x88b5'` on the HIGH segment with `tcpdump` while injecting one tagged frame on LOW, then swap the two and confirm nothing arrives (Handbook §4.7 has both commands).

4 Route B — the hardware diode

This is the three-converter arrangement. It needs no fiber splitter, works with every model tested, and is the one to build if you are unsure.

4.1 Why three converters

A media converter bridges copper Ethernet and fiber. It has two *separate* fiber ports, TX and RX. Left with nothing on its RX port, a converter sees no carrier, declares the fiber link down, and stops forwarding — so you cannot simply omit the return fiber. The trick is to give the sender light that carries *no data from the other side*. A third converter, its copper port deliberately left unplugged, supplies exactly that: carrier with nothing on it.

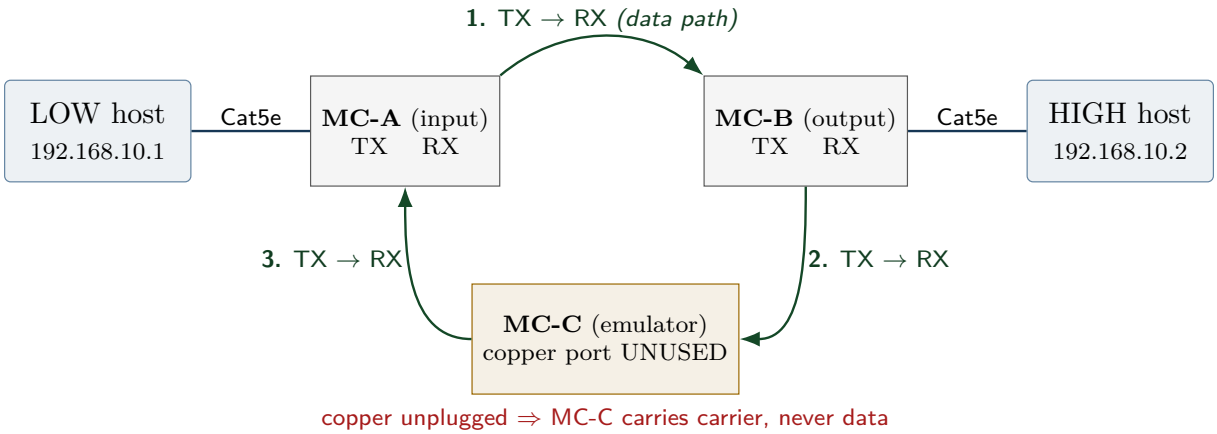

Figure 2: Three-converter diode. Data travels only along link 1. Links 2 and 3 exist solely so MC-A sees carrier on its RX port; because MC-C's copper port is unplugged, no data can traverse them.

The one wire that must stay unplugged

MC-C's *copper* port must be left disconnected. If anyone patches it into the LOW network, links 2 and 3 become a complete return path from HIGH to LOW and the diode is gone. Label that port, cover it, and put it on the commissioning checklist.

4.2 Bill of materials

Any dual-fiber media converter with separate TX and RX ports will do. These three are the ones this project has been built and tested with, and they cover fast Ethernet, gigabit multimode and gigabit single-mode.

Model	Speed	Fiber	λ	Conn.	Notes
TP-Link MC100CM	100 Mb/s	multi-mode	1310 nm	SC	Fast Ethernet, up to 2 km. Works in both topologies; 1310 nm splitters are catalogue parts.
TP-Link MC200CM	1 Gb/s	multi-mode	850 nm	SC	Gigabit, up to 550 m on OM2. Works in both topologies — but an 850 nm splitter is a special order with a long lead time.
TP-Link MC210CS	1 Gb/s	single-mode	1310 nm	SC	Gigabit, up to 15 km (check your revision). Cheap, commodity 1310 nm splitters.

Table 1: Distances are vendor figures for a normal link; a diode's fibers are a metre long, so see the note on attenuators below. Use three converters of the *same* model — mixing speeds or wavelengths across the triangle will not link up.

The triangle uses **three simplex fiber strands** — links 1, 2 and 3 in §4. Duplex patch cords carry two strands each, so two duplex cords cover it with one strand spare. Match the connector and fiber type to the converter, and budget one power supply per converter (9 V DC, centre-positive barrel). See Handbook §5.6 for power, UPS and rack notes.

Attenuators: only if you use long-range optics

Three *matched* converters need no attenuation, however short the fiber: within an optical class the maximum launch power is set at or below the maximum receive power, so a compliant transmitter cannot overload a compliant receiver. If instead you use a long-range converter or SFP — built to drive tens of kilometres — across the metre of fiber a diode needs, it can exceed the receiver's maximum input power, which shows up as CRC errors and dropped frames. Fixed attenuators (3 dB, 5 dB, 10 dB) are the fix; size them from the datasheet figures. Handbook §5.3.

4.3 Wiring

From	To	Why
LOW host NIC	MC-A copper	data enters the diode
MC-A fiber TX	**MC-B fiber RX**	**the one-way data path**
MC-B copper	HIGH host NIC	data leaves the diode
MC-B fiber TX	MC-C fiber RX	keeps MC-C's transceiver up
MC-C fiber TX	MC-A fiber RX	gives MC-A carrier so it holds link
MC-C copper	**nothing**	breaks the loop; *leave unplugged*
MC-A fiber RX	from MC-C only	must never come from MC-B

Power all three converters, then confirm each shows both a copper link LED and a fiber link LED (except MC-C, whose copper LED stays dark — that is correct). Now apply the *same* link-layer setup as in §3: addresses, `/etc/ethers`, `arp -f`, and the sysctl file. The firewall rules are optional here — the optics already enforce the direction — but applying them anyway is good defence in depth.

In practice

If MC-C goes completely dark with its copper port unplugged, your converters implement link-fault pass-through. Give MC-C a copper link that goes nowhere: an isolated switch port on a dead VLAN, or a loopback plug. Never give it a port on the LOW network.

5 Prove it is one-way

Do not skip this. Run all five, and record the output — this is your commissioning evidence.

1. **Physical trace.** Follow every fiber by hand against the wiring table. Confirm MC-C's copper port is unplugged.
2. **Carrier independence.** Unplug MC-B's copper cable from the HIGH host. MC-A must *keep* its fiber link LED. If MC-A goes down, its carrier is coming from the far end and you have built a bidirectional link.
3. **Reverse probe.** On the LOW host, start a capture that accepts everything:

   ```
   # LOW host -- must stay silent for the whole test
   tcpdump -ni enp1s0 -e -vv
   ```

   From the HIGH host, try hard to get a packet back:

   ```
   # HIGH host
   ping    -c 5           192.168.10.1
   arping  -c 5 -I enp1s0 192.168.10.1
   nc -u -w2              192.168.10.1 9999 <<< "probe"
   hping3 -S -c 5 -p 22   192.168.10.1        # TCP SYN
   ```

   The LOW capture must show **nothing** from the HIGH host. Every command above must fail or time out.
4. **TCP impossibility.** From the HIGH host, `nc -w 5 192.168.10.1 22` must always time out. A three-way handshake cannot complete across a diode; if it does, you have a bidirectional link.
5. **Forward path works.** From the LOW host, send a datagram and see it arrive:

   ```
   # HIGH host
   tcpdump -ni enp1s0 udp port 6514
   # LOW host
   echo hello | nc -u -w1 192.168.10.2 6514
   ```

There is no remote diagnostic

Unmanaged converters cannot tell you they have failed, and nothing can report back across the diode. Schedule a physical inspection — the Handbook §6.5 has a checklist.

6 Run the daemons

Start the HIGH side first, so nothing is lost:

```
# HIGH host -- recover, authenticate, re-emit to the local rsyslog
./bin/ld_deamp 6514 127.0.0.1 514 --key 9f2c...4ab1
```

Then the LOW side:

```
# LOW host -- take syslog on :1514, ship it across the diode
./bin/ld_amp 1514 192.168.10.2 6514 --key 9f2c...4ab1 \
             --batch 8 --parity 6 --interleave 8 --flush-ms 500
```

Point your senders at the amplifier. In `/etc/rsyslog.conf` on any host that produces logs:

```
*.*  @192.168.10.1:1514
```

and on the HIGH host, make rsyslog listen locally for what ld_deamp re-emits:

```
module(load="imudp")
input(type="imudp" address="127.0.0.1" port="514")
```

Listing 9: `/etc/rsyslog.d/10-diode-in.conf` — HIGH host

Test end to end:

```
# LOW host
logger -n 127.0.0.1 -P 1514 -p local0.crit "diode commissioning test"
# HIGH host
tail -f /var/log/syslog | grep "diode commissioning test"
```

6.1 The four options that matter

Option	Default	What it buys
`-batch N`	1	Packs N log lines into one Reed–Solomon block. At `-batch 1` the code degenerates to plain repetition (the paper's scheme); $N \geq 2$ gives real coding gain. Costs: a decode failure loses the whole batch.
`-parity M`	3	M parity fragments. Any K of $K + M$ fragments rebuild the message. Raise it when the link is lossy.
`-interleave D`	1	Spreads the fragments of D messages in time, so a burst cannot take every copy of one message. This is the fix for the paper's back-to-back repetition.
`-min-severity S`	7	Forward only severity $\leq S$. `-min-severity 4` ships warnings and worse. Unparseable lines are forwarded, not dropped.

Start at `-batch 8 -parity 6 -interleave 8`. If the HIGH side is missing messages, raise `-parity` first, then `-interleave`. Handbook §7.4 works through the tuning.

6.2 As services

```
[Unit]
Description=log-diode amplifier (LOW side)
After=network-online.target diode-link.service
Wants=network-online.target

[Service]
Type=simple
ExecStart=/opt/log-diode/bin/ld_amp 1514 192.168.10.2 6514 \
          --key ${DIODE_KEY} --batch 8 --parity 6 --interleave 8
EnvironmentFile=/etc/log-diode/key.env
Restart=always
```

```
RestartSec=2
DynamicUser=yes
AmbientCapabilities=CAP_NET_BIND_SERVICE
NoNewPrivileges=yes
ProtectSystem=strict
ProtectHome=yes
PrivateTmp=yes

[Install]
WantedBy=multi-user.target
```

Listing 10: `/etc/systemd/system/ld-amp.service` — LOW host

```
install -d -m 0700 /etc/log-diode
printf 'DIODE_KEY=%s\n' "$(openssl rand -hex 32)" > /etc/log-diode/key.env
chmod 0400 /etc/log-diode/key.env
```

The HIGH unit is the same shape with `ExecStart=.../ld_deamp 6514 127.0.0.1 514 -key ${DIODE_KEY}` and the identical key file, carried across by hand.

7 Where to go next

- **The two-converter build.** An ordinary 1×2 50:50 splitter replaces the third converter, and works at every wavelength here.[1] What differs is procurement — 1310 nm splitters are catalogue stock, 850 nm ones are a special order. Handbook §5.2–5.3.
- **Embedded and IoT.** Between two microcontrollers the diode is three wires: SPI with the MISO conductor omitted, optionally through optocouplers so each line is unidirectional in hardware. Handbook §5.8 — including why the *sender* must be the bus master, and why I^2C can never be a diode.
- **Threat model.** What a diode does and does not protect against, including the insider and covert-channel discussion — Handbook §2.
- **iptables and pf.** Full equivalents of the nftables ruleset, Handbook §4.
- **The layer-2 tunnel in full.** The complete `tapdiode.c`, its design notes, the bridge and sysctl hardening, and how to verify it at layer 2 — Handbook §4.7.
- **Why the daemons are proved.** The parser on the HIGH side chews bytes chosen by whoever compromised the LOW network, on a link with no way to ask for a retry — Handbook §7.1 and `docs/ASSURANCE.md`.
- **Scaling.** Several diodes in parallel, round-robin — Handbook §5.7.

[1] The *Sensors* paper states that this approach "did not work" for the MC200CM, and reports the three-converter arrangement being used instead (Anton et al., §2.2). That wording describes the parts available at the time rather than a limitation of the method: no 850 nm multimode splitter could be obtained within the timeframe of that work. The one eventually used had to be made to special order and took months to arrive, and with it in hand the two-converter topology works on the MC200CM exactly as it does at 1310 nm.